

Databricks Unity AI Gateway

A Detailed Guide to Enterprise AI Governance on Databricks

Prepared for Sourav Bhattacharya

Created: July 7, 2026, 14:25 UTC

Contents

1. What Is Unity AI Gateway?	3
1.1 Why It Exists	3
1.2 Where It Fits in the Databricks Platform	3
2. The Three Governance Layers	5
2.1 Layer 1 — AI Assets (Unity Catalog)	5
2.2 Layer 2 — AI Traffic (Unity AI Gateway)	5
2.3 Layer 3 — AI Service Behavior (Service Policies)	6
3. How a Request Actually Flows	7
4. Routing and Managing AI Traffic	8
4.1 Rate Limits	8
4.2 Traffic Splitting and Fallback Routing	8
4.3 Budgets and Hard Spend Caps	9
4.4 Smart Routing	9
5. Guardrails and Service Policies in Practice	10
5.1 Built-in Guardrails	10
5.2 Custom and Contextual Service Policies	10
5.3 Worked Example: Governing a Coding Agent's GitHub Access	10
6. Monitoring, Observability, and Cost Attribution	12
6.1 Usage Tracking	12
6.2 Cost Observability	12
6.3 Inference Tables (Audit Logging)	12
6.4 Unified Agent Tracing	12
6.5 Security Investigation with Lakewatch	12
6.6 Open Ecosystem Integrations	12
7. Working with Unity AI Gateway from Python	13
7.1 Calling a Governed Model Service	13
7.2 Handling a Denied or Transformed Response	13
7.3 Reading Usage Data with the Databricks SDK	14

8. Key Use Cases	16
9. Getting Started (Administrator Path)	16
10. Relationship to the Previous “AI Gateway”	17
11. Current Limitations to Keep in Mind	17
12. Summary	18
References	19

1. What Is Unity AI Gateway?

Unity AI Gateway is Databricks' governance solution for enterprise AI. Built on top of **Unity Catalog** (the same governance system Databricks uses for tables, volumes, and other data assets), it extends governance beyond static data and AI assets into the *runtime* interactions between models, agents, MCP (Model Context Protocol) servers, and tools.

In simple terms: every time a person, an application, or an autonomous agent calls a language model, invokes a tool over MCP, or triggers another agent, that request can be routed through Unity AI Gateway. The gateway decides whether the call is allowed, tracks what it costs, watches for risky content, and records what happened — all from one central place, regardless of whether the model or tool is hosted by Databricks or by an external provider such as OpenAI, Anthropic, or Google.

As of mid-2026, Unity AI Gateway is in **Beta**. Account administrators enable it from the account console's Previews page before workspace users can configure it.

1.1 Why It Exists

Modern AI systems are rarely a single model call. A typical enterprise agent might:

1. Accept a prompt from a user or another agent.
2. Call a foundation model (Databricks-hosted or external).
3. Use MCP tools to read a Slack channel, query a database, or open a GitHub pull request.
4. Hand off to a second agent for a sub-task.
5. Write a result back to a business system.

Each of these hops is a place where cost can spiral, sensitive data can leak, or an agent can take an action nobody approved. Historically, teams governed model access, tool access, and spend with different, disconnected systems — or with no formal governance at all. Unity AI Gateway consolidates all of it into a single control plane that reuses the access model teams already understand from data governance.

1.2 Where It Fits in the Databricks Platform

Unity AI Gateway sits between callers (users, coding agents, custom applications) and everything they call (models, MCP servers, other agents). It relies on two companion concepts:

- **Unity Catalog** — registers models, MCP services, agents, functions, and connections as *securable objects*, governed with familiar grant/revoke privileges.
- **Service policies** — attribute-based access control (ABAC) rules scoped to AI services that inspect the *content* of a request or response and allow, deny, transform, or require human approval for it.

Unity AI Gateway: Three Governance Layers

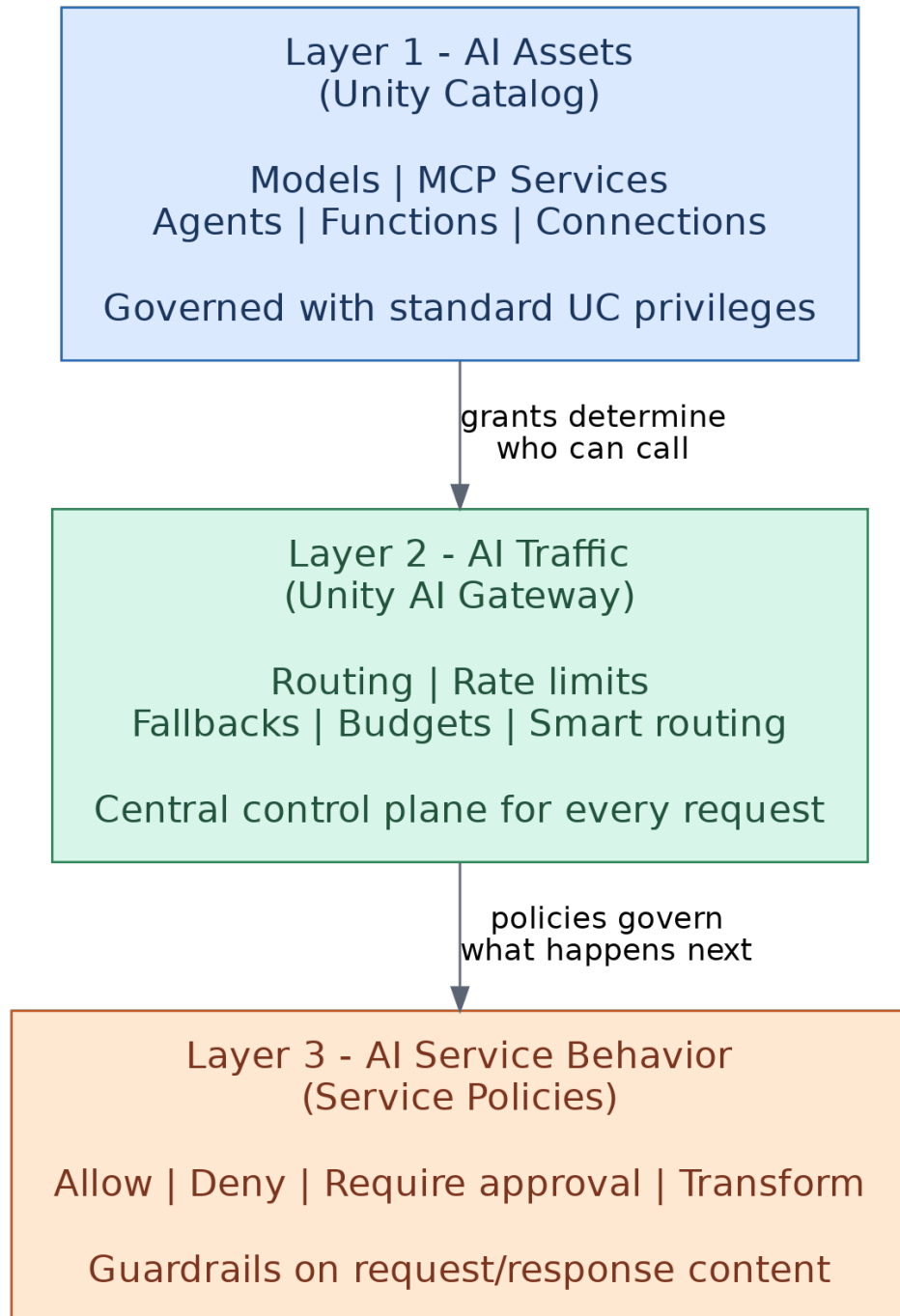


Figure 1: Unity AI Gateway sits across three cooperating governance layers.

2. The Three Governance Layers

Unity AI Gateway's model can be understood as three layers stacked on top of each other. Each layer answers a different question.

2.1 Layer 1 — AI Assets (Unity Catalog)

Question answered: *Who is allowed to use this model, tool, or agent at all?*

Unity Catalog manages the following as securable objects, governed with the same privileges (GRANT, REVOKE) used for tables and volumes:

- **Models** — registered ML models, including Databricks-hosted foundation models available through Foundation Model APIs.
- **MCP tools / MCP Services** — MCP servers registered in Unity Catalog, with per-tool filtering. Databricks provides managed MCP services out of the box for common apps such as Google Drive, Jira, Confluence, Slack, GitHub, and SharePoint, plus support for registering custom MCP servers.
- **Agents** — AI agents registered and governed alongside tables, models, and functions.
- **Connections** — Unity Catalog HTTP connections used to reach external APIs and MCP servers.
- **Functions** — Unity Catalog functions used as agent tools or data transformations.
- **Skills** (newer addition) — a governed, discoverable inventory of reusable agent skills.

Because these are ordinary Unity Catalog objects, existing patterns apply directly: a data platform team can grant a marketing team EXECUTE on a specific model but not on an internal HR model, exactly as they would restrict access to a sensitive table.

2.2 Layer 2 — AI Traffic (Unity AI Gateway)

Question answered: *How is this request routed, and what does it cost?*

Once a caller is authorized to use an asset, Unity AI Gateway becomes the runtime control plane for the actual traffic:

- Routes requests to model services and MCP services from a single place.
- Applies **rate limits** (queries per minute / tokens per minute) per user, group, or service principal.
- Configures **traffic splitting and fallback routing** across multiple backends for availability.
- Enforces **budgets and hard spend caps** — stopping requests once a threshold is hit, rather than only alerting after the money is spent.
- Performs **smart routing**, recommending or automatically selecting a model based on task complexity, quality needs, and cost.

2.3 Layer 3 — AI Service Behavior (Service Policies)

Question answered: *Given that this call is allowed and routed, should this specific request or response be permitted, based on its content?*

This is where **service policies** operate. A service policy is an ABAC policy scoped to AI services. It can:

- **Allow** the interaction.
- **Deny** the interaction outright.
- **Require human approval** before proceeding (a human-in-the-loop checkpoint).
- **Transform** the request or response — for example, redact personally identifiable information (PII) using a built-in policy such as `system.ai.mask_pii`.

Built-in guardrails ship for common risks: PII exposure, prompt injection, jailbreak attempts, and unsafe content. Organizations can also write and attach custom policy functions, or integrate third-party AI security vendors into this layer (see Section 6).

3. How a Request Actually Flows

The diagram below traces a single call end to end — for example, a developer’s coding agent asking to open a GitHub pull request through a managed MCP service.

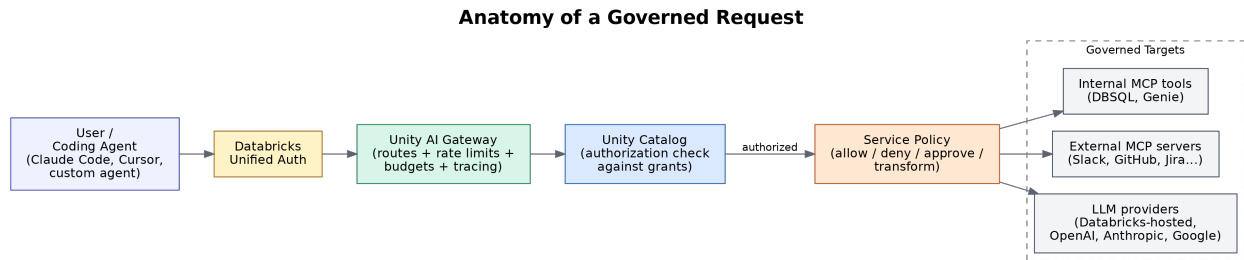


Figure 2: A request from a user or coding agent passes through authentication, gateway routing, Unity Catalog authorization, and service-policy evaluation before reaching its target.

Step by step:

1. **Caller** — A user, or an agent acting on the user’s behalf (e.g., Claude Code, Cursor, Codex, Gemini CLI, or a custom agent), issues a request.
2. **Databricks Unified Auth** — The request is authenticated so that Databricks knows exactly which identity is making the call, even when the call originates from a third-party coding agent.
3. **Unity AI Gateway** — The request enters the control plane, which will apply rate limits, budget checks, and routing logic, and record the interaction for tracing.
4. **Unity Catalog authorization** — The gateway checks whether the calling identity actually has the Unity Catalog privileges needed for the target model, MCP service, or agent.
5. **Service policy evaluation** — If authorized, any attached service policy inspects the request content and decides to allow, deny, transform, or route it for human approval.
6. **Target system** — Only after clearing all of the above does the request reach its destination: an internal MCP tool (like DBSQL or Genie), an external MCP server (Slack, GitHub, Jira, etc.), or an LLM provider (Databricks-hosted or external).

This means the same enforcement point governs *internal* Databricks resources and *external* systems identically — a coding agent calling an external GitHub MCP server is governed exactly like an internal call to a Databricks SQL warehouse.

4. Routing and Managing AI Traffic

Unity AI Gateway’s traffic-management features exist to answer a very practical business problem: as AI usage scales across many teams, capacity and cost can spin out of control unless someone can see and control it centrally.

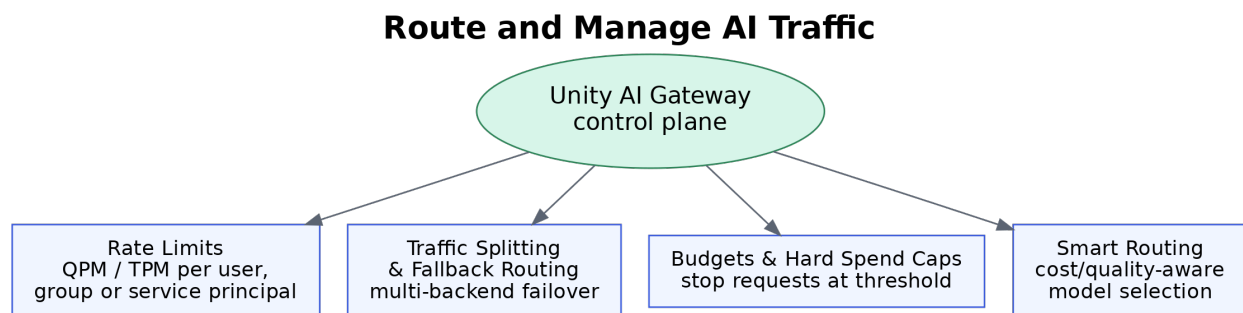


Figure 3: Unity AI Gateway centralizes rate limiting, fallback routing, budget enforcement, and smart routing.

4.1 Rate Limits

Rate limits can be set as queries-per-minute (QPM) or tokens-per-minute (TPM), scoped to:

- An entire model service or MCP service.
- A specific user.
- A group.
- A service principal (i.e., an application or automated pipeline).

Example use case: A shared “gpt-4-class” model service is used by ten teams. Without limits, one runaway batch job could exhaust capacity for everyone else. An admin sets a per-service-principal QPM cap on the batch job’s identity, leaving headroom for interactive users.

4.2 Traffic Splitting and Fallback Routing

A single logical model service can route across multiple physical backends:

- Split traffic (e.g., 80% to a primary model, 20% to a cheaper model for A/B comparison).
- Automatically fail over to a backup model if the primary is unavailable, rate-limited, or degraded.

This raises availability without requiring application code to know about multiple providers — the failover logic lives in the gateway configuration, not in every calling application.

4.3 Budgets and Hard Spend Caps

Unlike alert-only cost dashboards, Unity AI Gateway can enforce **hard spend caps**: once a budget threshold is reached, further requests are stopped automatically rather than merely flagged after the fact. Budgets can be scoped per user, team, tool, or use case, giving fine-grained attribution of where AI spend is going and why.

4.4 Smart Routing

Smart routing evaluates production traffic to recommend, or automatically select, the most appropriate model for a given request — balancing response quality against cost and latency. This lets an organization compare multiple providers on real traffic and shift volume toward whichever model is performing best for the money.

5. Guardrails and Service Policies in Practice

Service policies are how Unity AI Gateway enforces *content-aware* rules — decisions that depend on what is actually inside a request or response, not just who is asking.

5.1 Built-in Guardrails

Databricks ships built-in protections that can be attached to a model service or MCP service without writing custom code:

- **PII detection and masking** — e.g., the `system.ai.mask_pii` policy redacts personal data before it reaches a model or before a response reaches the caller.
- **Prompt injection detection** — flags or blocks inputs designed to hijack an agent’s instructions.
- **Jailbreak detection** — flags attempts to bypass a model’s safety behavior.
- **Unsafe content filtering** — blocks disallowed categories of content in requests or responses.

5.2 Custom and Contextual Service Policies

Beyond the built-ins, organizations can write their own service policy functions and attach them to a Model Service or MCP Service. A newer capability, **Contextual Service Policies** (introduced in Beta at Data + AI Summit 2026), lets policies key off richer context:

- The identity of the user or agent.
- Which model, MCP service, or specific tool is being invoked.
- The content of the request and response.

Illustrative examples of contextual policies:

- Require human approval before a coding agent pushes code to GitHub.
- Deny write access to a sensitive Google Drive folder for a given agent identity.
- Block any request or response containing regulated data categories (e.g., financial account numbers).

5.3 Worked Example: Governing a Coding Agent’s GitHub Access

Databricks documents an end-to-end tutorial for exactly this scenario: an engineering org wants developers to use an external coding agent (say, Claude Code) against a GitHub MCP service, but wants to restrict what the agent can do on GitHub and require a human check before anything is pushed.

At a conceptual level the setup looks like this:

1. Register the GitHub MCP server as a Unity Catalog **MCP Service**.
2. Grant the engineering group EXECUTE privilege on that MCP Service (Layer 1 — assets).
3. Configure a rate limit on the service so no single agent session can spam the GitHub API (Layer 2 — traffic).

4. Attach a service policy that requires approval specifically for tool calls that write to the repository, such as `create_pull_request` or `push`, while allowing read-only calls like `list_issues` to pass straight through (Layer 3 — behavior).

The result: developers get full read access to GitHub through their coding agent immediately, while every write action pauses for a human decision, and every call — read or write — is logged for audit.

6. Monitoring, Observability, and Cost Attribution

Because every governed call passes through the gateway, Unity AI Gateway can produce a single, consistent telemetry stream instead of stitched-together logs from many systems.

6.1 Usage Tracking

System tables record requests, token usage, and latency for model services, queryable like any other Unity Catalog table.

6.2 Cost Observability

Databricks cost can be attributed to specific services, target models, calling principals, and custom tags — turning “our AI bill went up” into “team X’s agent Y increased spend on model Z by N%.”

6.3 Inference Tables (Audit Logging)

Full request and response payloads can be logged to Unity Catalog Delta tables, supporting debugging, compliance audits, and incident investigation.

6.4 Unified Agent Tracing

Unity AI Gateway captures both model interactions and MCP tool activity in a single governed tracing layer, integrated with **MLflow** for trace visualization and evaluation. This answers questions like: for a single agent session, exactly which model calls and tool calls happened, in what order, and what did each one cost?

6.5 Security Investigation with Lakewatch

Traces can be streamed into **Databricks Lakewatch** (Databricks’ security/observability product) to detect suspicious activity, investigate policy violations, and support broader AI security investigations alongside other enterprise telemetry.

6.6 Open Ecosystem Integrations

Unity AI Gateway is designed as an open platform rather than a closed box. Databricks has announced integrations with third-party AI security, identity, and data-protection vendors — including CrowdStrike, Palo Alto Networks, Zscaler, Okta, SailPoint, Saviynt, Cyera, HiddenLayer, and others — so that specialized security tooling can plug directly into the gateway’s runtime path.

7. Working with Unity AI Gateway from Python

Because Databricks-hosted foundation models exposed through Unity AI Gateway are OpenAI-API-compatible, most existing Python tooling works with only a base URL and token change. The examples below use Python, as is conventional for Databricks AI workloads.

7.1 Calling a Governed Model Service

```
import os
from openai import OpenAI

# The gateway-governed endpoint for a registered model service.
# Replace with your workspace URL and the model service's serving path.
client = OpenAI(
    base_url="https://<your-workspace-host>/serving-endpoints",
    # a personal access token or OAuth token
    api_key=os.environ["DATABRICKS_TOKEN"],
)

system_msg = "You are a concise, factual assistant."
user_msg = "Summarize what Unity AI Gateway does in two sentences."

response = client.chat.completions.create(
    model="my-governed-chat-model", # UC-registered model service name
    messages=[
        {"role": "system", "content": system_msg},
        {"role": "user", "content": user_msg},
    ],
    max_tokens=200,
)

print(response.choices[0].message.content)
```

Every call made this way is subject to whatever rate limits, budgets, and service policies an administrator has attached to my-governed-chat-model — the calling application does not need to implement any of that logic itself.

7.2 Handling a Denied or Transformed Response

Because a service policy can deny a request or redact content, calling code should be written defensively:

```
from openai import OpenAI, APIStatusError

client = OpenAI(
    base_url="https://<your-workspace-host>/serving-endpoints",
    api_key=os.environ["DATABRICKS_TOKEN"],
```

```

)

def ask_model(prompt: str) -> str:
    try:
        response = client.chat.completions.create(
            model="my-governed-chat-model",
            messages=[{"role": "user", "content": prompt}],
        )
        return response.choices[0].message.content
    except APIStatusError as e:
        # A service policy denial or a rate-limit breach typically
        # surfaces as a 4xx status with a policy-specific error body.
        if e.status_code == 403:
            return "Request blocked by a Unity AI Gateway service policy."
        if e.status_code == 429:
            return "Rate limit exceeded – retry after backoff."
        raise

print(ask_model("What is our Q3 customer churn rate?"))

```

7.3 Reading Usage Data with the Databricks SDK

Because usage and cost data lands in governed Unity Catalog tables, it can be queried like any other table — for example, from a notebook using the Databricks SQL connector:

```

from databricks import sql
import os

connection = sql.connect(
    server_hostname=os.environ["DATABRICKS_SERVER_HOSTNAME"],
    http_path=os.environ["DATABRICKS_HTTP_PATH"],
    access_token=os.environ["DATABRICKS_TOKEN"],
)

query = """
SELECT
    requesting_principal,
    model_service_name,
    SUM(token_count) AS total_tokens,
    SUM(estimated_cost_usd) AS total_cost
FROM system.ai_gateway.usage_logs
WHERE request_date >= current_date() - INTERVAL 7 DAYS
GROUP BY requesting_principal, model_service_name
ORDER BY total_cost DESC
LIMIT 20
"""

```

```
with connection.cursor() as cursor:  
    cursor.execute(query)  
    for row in cursor.fetchall():  
        print(row)  
  
connection.close()
```

(Table and column names are illustrative — consult current Databricks documentation for exact system table schemas, as these are evolving during the Beta.)

8. Key Use Cases

Use case	What Unity AI Gateway provides
Agent governance	Centralized visibility, policy enforcement, and auditability as more teams build and deploy agents.
Coding agent governance	One governed layer across multiple coding agents (Claude Code, Cursor, Codex, Gemini CLI) and providers, instead of per-tool sprawl.
MCP governance	Agents inherit the calling user's permissions, are limited to approved tools, and every tool call is auditable.
Model governance	Compare quality, cost, and latency across providers on real traffic; route intelligently; fail over automatically.
AI spend governance	Real-time spend limits and hard caps across every provider, with usage broken out by user, team, agent, and application.

9. Getting Started (Administrator Path)

Databricks documents a five-step rollout for an administrator setting this up for the first time:

1. **Enable the AI governance previews** — an account admin turns on Unity AI Gateway and service policies from the account console's Previews page (Beta features are opt-in).
2. **Govern access to AI assets in Unity Catalog** — register models and MCP Services as securables, and grant only the privileges each principal actually needs.
3. **Route and control AI traffic** — create model services, point coding agents at a governed model service, and configure rate limits and spend caps.
4. **Control request/response content with service policies** — attach built-in or custom policies to allow, deny, transform, or require approval.
5. **Monitor usage and cost** — confirm governance is working using usage tables, cost observability views, and inference tables.

Two hands-on tutorials Databricks publishes for a first working example:

- *Govern a coding agent's GitHub MCP access.*
- *Moderate a model service's content with guardrails and service policies.*

10. Relationship to the Previous “AI Gateway”

Databricks previously shipped a workspace-level **AI Gateway for serving endpoints**, which added usage tracking, payload logging, rate limits, and guardrails directly onto individual model serving endpoints (external model endpoints, Foundation Model API endpoints, and custom model endpoints). Unity AI Gateway is the newer, broader evolution of that idea: instead of being configured per-endpoint at the workspace level, it is a Unity Catalog-integrated, account-wide control plane covering models, MCP services, and agents together. Existing serving-endpoint-level AI Gateway configuration continues to be documented and supported alongside the newer Unity AI Gateway.

11. Current Limitations to Keep in Mind

- **Beta status:** Unity AI Gateway and service policies are in Beta as of mid-2026; features, table schemas, and UI surfaces are subject to change, and an account admin must explicitly enable the preview.
- **No charge during Beta:** Databricks states Unity AI Gateway features do not incur separate charges while in Beta (underlying model serving and compute costs still apply).
- **Requires Unity Catalog:** A workspace must already be enabled for Unity Catalog before Unity AI Gateway concepts apply.

12. Summary

Unity AI Gateway gives Databricks customers one place to answer three questions for every AI interaction in the organization: *who* is allowed to make this call, *how* is the resulting traffic routed and paid for, and *what* is actually allowed to happen given the content of the request. By reusing Unity Catalog’s existing privilege model for the “who,” adding a dedicated routing and cost-control plane for the “how,” and layering attribute-based service policies for the “what,” it turns AI governance from a patchwork of point solutions into a single, auditable control plane — covering Databricks-hosted models, external providers, internal tools, external MCP servers, and third-party coding agents alike.

References

1. Databricks. *AI governance with Unity AI Gateway* (AWS docs). <https://docs.databricks.com/aws/en/ai-gateway/>
2. Databricks. *Get started with AI governance*. <https://docs.databricks.com/aws/en/ai-gateway/get-started>
3. Databricks. *Unity AI Gateway: Multi-AI governance and cost control* (product page). <https://www.databricks.com/product/artificial-intelligence/unity-ai-gateway>
4. Databricks Blog. *AI governance at Data + AI Summit 2026: What's new with Unity AI Gateway* (June 16, 2026). <https://www.databricks.com/blog/ai-governance-data-ai-summit-2026-whats-new-unity-ai-gateway>
5. Databricks. *AI governance with Unity AI Gateway* (Azure docs / Microsoft Learn). <https://learn.microsoft.com/en-us/azure/databricks/ai-gateway/>
6. Databricks. *Configure AI Gateway on model serving endpoints*. <https://docs.databricks.com/aws/en/ai-gateway/configure-ai-gateway-endpoints>
7. Databricks. *Service policies for AI securables*. <https://docs.databricks.com/aws/en/data-governance/unity-catalog/service-policies>
8. Databricks. *AI Gateway for serving endpoints (previous version)*. <https://docs.databricks.com/aws/en/ai-gateway/overview-serving-endpoints>